Moser's entropy compression argument

Terence Tao · 5 August 2009

https://terrytao.wordpress.com/2009/08/05/mosers-entropy-compression-argument/

tao2typ · converted to Typst 2026-10-14 (UTC)

There are many situations in combinatorics in which one is running some sort of iteration algorithm to continually "improve" some object A; each loop of the algorithm replaces A with some better version A' of itself, until some desired property of A is attained and the algorithm halts. In order for such arguments to yield a useful conclusion, it is often necessary that the algorithm halts in a finite amount of time, or (even better), in a bounded amount of time. (In general, one cannot use infinitary iteration tools, such as transfinite induction or Zorn's lemma, in combinatorial settings, because the iteration processes used to improve some target object A often degrade some other finitary quantity B in the process, and an infinite iteration would then have the undesirable effect of making B infinite.)

A basic strategy to ensure termination of an algorithm is to exploit a *monotonicity property*, or more precisely to show that some key quantity keeps increasing (or keeps decreasing) with each loop of the algorithm, while simultaneously staying bounded. (Or, as the economist Herbert Stein was fond of saying, "If something cannot go on forever, it must stop.")

Here are four common flavours of this monotonicity strategy:

- The *mass increment argument*. This is perhaps the most familiar way to ensure termination: make each improved object A' "heavier" than the previous one A by some non-trivial amount (e.g. by ensuring that the cardinality of A' is strictly greater than that of A, thus $|A'| \geq |A| + 1$). Dually, one can try to force the amount of "mass" remaining "outside" of A in some sense to decrease at every stage of the iteration. If there is a good upper bound on the "mass" of A that stays essentially fixed throughout the iteration process, and a lower bound on the mass increment at each stage, then the argument terminates. Many "greedy algorithm" arguments are of this type. The proof of the Hahn decomposition theorem in measure theory also falls into this category. The general strategy here is to keep looking for useful pieces of mass outside of A, and add them to A to form A', thus exploiting the additivity properties of mass. Eventually no further usable mass remains to be added (i.e. A is *maximal* in some L^1 sense), and this should force some desirable property on A.
- The *density increment argument*. This is a variant of the mass increment argument, in which one increments the "density" of A rather than the "mass". For instance, A might be contained in some ambient space P, and one seeks to improve A to A' (and P to P') in such a way that the density of the new object in the new ambient space is better than that of the previous object (e.g. $|A'|/|P'| \geq |A|/|P| + c$ for some $c > 0$). On the other hand, the density of A is clearly bounded above by 1. As long as one has a sufficiently good lower bound on the density increment at each stage, one can conclude an upper bound on the number of iterations in the algorithm. The prototypical example of this is Roth's proof of his theorem that every set of integers of positive upper density contains an arithmetic progression of length three. The general strategy here is to keep looking for useful density fluctuations inside A, and then "zoom in" to a region of increased density by reducing A and P appropriately. Eventually no further usable density fluctuation remains (i.e. A is *uniformly distributed*), and this should force some desirable property on A.
- The *energy increment argument*. This is an "L^2" analogue of the "L^1"-based mass increment argument (or the "L^∞"-based density increment argument), in which one seeks to increments the amount of "energy" that A captures from some reference object X, or (equivalently) to decrement the amount of energy of X which is still "orthogonal" to A. Here A and X are related somehow to a Hilbert space, and the energy involves the norm on that space. A classic example of this type

of argument is the existence of orthogonal projections onto closed subspaces of a Hilbert space; this leads among other things to the construction of conditional expectation in measure theory, which then underlies a number of arguments in ergodic theory, as discussed for instance in this earlier blog post. Another basic example is the standard proof of the Szemerédi regularity lemma (where the "energy" is often referred to as the "index"). These examples are related; see this blog post for further discussion. The general strategy here is to keep looking for useful pieces of energy orthogonal to A, and add them to A to form A', thus exploiting square-additivity properties of energy, such as Pythagoras' theorem. Eventually, no further usable energy outside of A remains to be added (i.e. A is *maximal* in some L^2 sense), and this should force some desirable property on A.

- The *rank reduction argument*. Here, one seeks to make each new object A' to have a lower "rank", "dimension", or "order" than the previous one. A classic example here is the proof of the linear algebra fact that given any finite set of vectors, there exists a linearly independent subset which spans the same subspace; the proof of the more general Steinitz exchange lemma is in the same spirit. The general strategy here is to keep looking for "collisions" or "dependencies" within A, and use them to collapse A to an object A' of lower rank. Eventually, no further usable collisions within A remain, and this should force some desirable property on A.

Much of my own work in additive combinatorics relies heavily on at least one of these types of arguments (and, in some cases, on a nested combination of two or more of them). Many arguments in nonlinear partial differential equations also have a similar flavour, relying on various *monotonicity formulae* for solutions to such equations, though the objective in PDE is usually slightly different, in that one wants to keep control of a solution as one approaches a singularity (or as some time or space coordinate goes off to infinity), rather than to ensure termination of an algorithm. (On the other hand, many arguments in the theory of *concentration compactness*, which is used heavily in PDE, does have the same algorithm-terminating flavour as the combinatorial arguments; see this earlier blog post for more discussion.)

Recently, a new species of monotonicity argument was introduced by Moser, as the primary tool in his elegant new proof of the Lovász local lemma. This argument could be dubbed an *entropy compression argument*, and only applies to probabilistic algorithms which require a certain collection R of random "bits" or other random choices as part of the input, thus each loop of the algorithm takes an object A (which may also have been generated randomly) and some portion of the random string R to (deterministically) create a better object A' (and a shorter random string R', formed by throwing away those bits of R that were used in the loop). The key point is to design the algorithm to be partially *reversible*, in the sense that given A' and R' and some additional data H' that logs the cumulative *history* of the algorithm up to this point, one can reconstruct A together with the remaining portion R not already contained in R'. Thus, each stage of the argument *compresses* the information-theoretic content of the string $A + R$ into the string $A' + R' + H'$ in a lossless fashion. However, a random variable such as $A + R$ cannot be compressed losslessly into a string of expected size smaller than the Shannon entropy of that variable. Thus, if one has a good lower bound on the entropy of $A + R$, and if the length of $A' + R' + H'$ is significantly less than that of $A + R$ (i.e. we need the marginal growth in the length of the history file H' per iteration to be less than the marginal amount of randomness used per iteration), then there is a limit as to how many times the algorithm can be run, much as there is a limit as to how many times a random data file can be compressed before no further length reduction occurs.

It is interesting to compare this method with the ones discussed earlier. In the previous methods, the failure of the algorithm to halt led to a new iteration of the object A which was "heavier", "denser", captured more "energy", or "lower rank" than the previous instance of A. Here, the failure of the algorithm to halt leads to new information that can be used to "compress" A (or more precisely, the full state $A + R$) into a smaller amount of space. I don't know yet of any application of this new type of termination strategy to the fields I work in, but one could imagine that it could eventually be of use (perhaps to

show that solutions to PDE with sufficiently "random" initial data can avoid singularity formation?), so I thought I would discuss it here.

Below the fold I give a special case of Moser's argument, based on a blog post of Lance Fortnow on this topic.

Rather than deal with the Lovász local lemma in full generality, I will follow Fortnow and work with a special case of this lemma involving the k-satisfiability problem (in conjunctive normal form). Here, one is given a set of *boolean variables* $x_1, \ldots, x_n$ together with their negations $\neg x_1, \ldots, \neg x_n$; we refer to the $2n$ variables and their negations collectively as *literals*. We fix an integer $k \geq 2$, and define a (length k) *clause* to be a disjunction of k literals, for instance

$$x_3 \vee \neg x_5 \vee x_9$$

is a clause of length three, which is true unless x_3 is false, x_5 is true, and x_9 is false. We define the *support* of a clause to be the set of variables that are involved in the clause, thus for instance $x_3 \vee \neg x_5 \vee x_9$ has support $\{x_3, x_5, x_9\}$. To avoid degeneracy we assume that no clause uses a variable more than once (or equivalently, all supports have cardinality exactly k), thus for instance we do not consider $x_3 \vee x_3 \vee x_9$ or $x_3 \vee \neg x_3 \vee x_9$ to be clauses.

Note that the failure of a clause reveals complete information about all k of the boolean variables in the support; this will be an important fact later on.

The *k-satisfiability problem* is the following: given a set S of clauses of length k involving n boolean variables $x_1, \ldots, x_n$, is there a way to assign truth values to each of the $x_1, \ldots, x_n$, so that all of the clauses are simultaneously satisfied?

For general S, this problem is easy for $k = 2$ (essentially equivalent to the problem of 2-colouring a graph), but NP-complete for $k \geq 3$ (this is the famous Cook-Levin theorem). But the problem becomes simpler if one makes some more assumptions on the set S of clauses. For instance, if the clauses in S have disjoint supports, then they can be satisfied independently of each other, and so one easily has a positive answer to the satisfiability problem in this case. (Indeed, one only needs each clause in S to have *one* variable in its support that is disjoint from all the other supports in order to make this argument work.)

Now suppose that the clauses S are not completely disjoint, but have a limited amount of overlap; thus *most* clauses in S have disjoint supports, but not all. With too much overlap, of course, one expects satisfability to fail (e.g. if S is the set of *all* length k clauses). But with a sufficiently small amount of overlap, one still has satisfiability:

> **Theorem 1 (Lovász local lemma, special case)** Suppose that S is a set of length k clauses, such that the support of each clause s in S intersects at most 2^{k-C} supports of clauses in S (including s itself), where C is a sufficiently large absolute constant. Then the clauses in S are simultaneously satisfiable.

One of the reasons that this result is powerful is that the bounds here are uniform in the number n of variables. Apart from the loss of C, this result is sharp; consider for instance the set S of all 2^k clauses with support $\{x_1, \ldots, x_k\}$, which is clearly unsatisfiable.

The standard proof of this theorem proceeds by assigning each of the n boolean variables $x_1, \ldots, x_n$ a truth value $a_1, \ldots, a_n \in \{\text{true}, \text{false}\}$ independently at random (with each truth value occurring with an equal probability of $1/2$); then each of the clauses in S has a positive zero probability of holding (in fact, the probability is $1 - 2^{-k}$). Furthermore, if E_s denotes the event that a clause $s \in S$ is satisfied, then the E_s are mostly independent of each other; indeed, each event E_s is independent of all but most 2^{k-C} other events $E_{s'}$. Applying the Lovász local lemma, one concludes that the E_s simultaneously hold with positive probability (if C is a little bit larger than $\log_2 e$), and the claim follows.

The textbook proof of the Lovász local lemma is short but nonconstructive; in particular, it does not easily offer any quick way to compute an actual satisfying assignment for $x_1, \ldots, x_n$, only saying

that such an assignment exists. Moser's argument, by contrast, gives a simple and natural algorithm to locate such an assignment (and thus prove Theorem 1). (The constant C becomes 3 rather than $\log_2 e$, although the $\log_2 e$ bound has since been recovered in a paper of Moser and Tardos.)

As with the usual proof, one begins by randomly assigning truth values $a_1, ..., a_n \in \{\text{true}, \text{false}\}$ to $x_1, ..., x_n$; call this random assignment $A = (a_1, ..., a_n)$. If A satisfied all the clauses in S, we would be done. However, it is likely that there will be some non-empty subset T of clauses in S which are not satisfied by A.

We would now like to modify A in such a manner to reduce the number $|T|$ of violated clauses. If, for instance, we could always find a modification A' of A whose set T' of violated clauses was strictly smaller than T (assuming of course that T is non-empty), then we could iterate and be done (this is basically a mass decrement argument). One obvious way to try to achieve this is to pick a clause s in T that is violated by A, and modify the values of A on the support of s to create a modified set A' that satisfies s, which is easily accomplished; in fact, any non-trivial modification of A on the support will work here. In order to maximize the amount of entropy in the system (which is what one wants to do for an entropy compression argument), we will choose this modification of A' *randomly*; in particular, we will use k fresh random bits to replace the k bits of A in the support of s. (By doing so, there is a small probability (2^{-k}) that we in fact do not change A at all, but the argument is (very) slightly simpler if we do not bother to try to eliminate this case.)

If all the clauses had disjoint supports, then this strategy would work without difficulty. But when the supports are not disjoint, one has a problem: every time one modifies A to "fix" a clause s by modifying the variables on the support of s, one may cause other clauses s' whose supports overlap those of s to fail, thus potentially increasing the size of T by as much as $2^{k-C} - 1$. One could then try fixing all the clauses which were broken by the first fix, but it appears that the number of clauses needed to repair could grow indefinitely with this procedure, and one might never terminate in a state in which all clauses are simultaneously satisfied.

The key observation of Moser, as alluded earlier, is that each failure of a clause s for an assignment A reveals k bits of information about A, namely that the exact values that A assigns to the support of s. The plan is then to use each failure of a clause as a part of a *compression protocol* that compresses A (plus some other data) losslessly into a smaller amount of space. A crucial point is that at each stage of the process, the clause one is trying to fix is almost always going to be one that overlapped the clause that one had just previously fixed. Thus the total number of possibilities for each clause, given the previous clauses, is basically 2^{k-C}, which requires only $k - C$ bits of storage, compared with the k bits of entropy that have been eliminated. This is what is going to force the algorithm to terminate in finite time (with positive probability).

Let's make the details more precise. We will need the following objects:

- A truth assignment A of n truth values $a_1, ..., a_n$, which is initially assigned randomly, but which will be modified as the algorithm progresses;
- A long random string R of bits, from which we will make future random choices, with each random bit being removed from R as it is read.

We also need a recursive algorithm $Fix(s)$, which modifies the string A to satisfy a clause s in S (and, as a bonus, may also make A obey some other clauses in S that it did not previously satisfy). It is defined recursively:

- Step 1. If A already satisfies s, do nothing (i.e. leave A unchanged).
- Step 2. Otherwise, read off k random bits from R (thus shortening R by k bits), and use these to replace the k bits of A on the support of s in the obvious manner (ordering the support of s by some fixed ordering, and assigning the jth bit from R to the jth variable in the support for $1 \leq j \leq k$).

- Step 3. Next, find all the clauses s' in S whose supports intersect s, and which A now violates; this is a collection of at most 2^{k-C} clauses, possibly including s itself. Order these clauses s' in some arbitrary fashion, and then apply $Fix(s')$ to each such clause in turn. (Thus the original algorithm $Fix(s)$ is put "on hold" on some CPU stack while all the child processes $Fix(s')$ are executed; once all of the child processes are complete, $Fix(s)$ then terminates also.)

An easy induction shows that if $Fix(s)$ terminates, then the resulting modification of A will satisfy s; and furthermore, any other clause s' in S which was already satisfied by A before $Fix(s)$ was called, will continue to be satisfied by A after $Fix(s)$ is called. Thus, $Fix(s)$ can only serve to decrease the number of unsatisfied clauses T in S, and so one can fix all the clauses by calling $Fix(s)$ once for each clause in T – provided that these algorithms all terminate.

Each time Step 2 of the Fix algorithm is called, the assignment A changes to a new assignment A', and the random string R changes to a shorter string R'. Is this process reversible? Yes – provided that one knows what clause s was being fixed by this instance of the algorithm. Indeed, if s, A', R' are known, then A can be recovered by changing the assignment of A' on the support of s to the only set of choices that violates s, while R can be recovered from R' by appending to R' the bits of A' on the support of s.

This type of reversibility does not seem very useful for an entropy compression argument, because while R' is shorter than R by k bits, it requires about $\log|S|$ bits to store the clause s. So the map $A + R \mapsto A' + R' + s$ is only a compression if $\log|S| < k$, which is not what is being assumed here (and in any case the satisfiability of S in the case $\log|S| < k$ is trivial from the union bound).

The key trick is that while it does indeed take $\log|S|$ bits to store any given clause s, there is an economy of scale: after many recursive applications of the fix algorithm, the *marginal* amount of bits needed to store s drops to merely $k - C + O(1)$, which is less than k if C is large enough, and which will therefore make the entropy compression argument work.

Let's see why this is the case. Observe that the clauses s for which the above algorithm $Fix(s)$ is called come in two categories. Firstly, there are those s which came from the original list T of failed clauses. Each of these will require $O(\log|S|)$ bits to store – but there are only $|T|$ of them. Since $|T| \leq |S|$, the net amount of storage space required for these clauses is $O(|S| \log|S|)$ at most. Actually, one can just store the subset T of S using $|S|$ bits (one for each element of S, to record whether it lies in T or not).

Of more interest is the other category of clauses s, in which $Fix(s)$ is called recursively from some previously invoked call $Fix(s')$ to the fix algorithm. But then s is one of the at most 2^{k-C} clauses in S whose support intersects that of s'. Thus one can encode s using s' and a number between 1 and 2^{k-C}, representing the position of s (with respect to some arbitrarily chosen fixed ordering of S) in the list of all clauses in S whose supports intersect that of s'. Let us call this number the *index* of the call $Fix(s)$.

Now imagine that while the Fix routine is called, a running *log file* (or history) H of the routine is kept, which records s each time one of the original $|T|$ calls $Fix(s)$ with $s \in T$ is invoked, and also records the index of any other call $Fix(s)$ made during the recursive procedure. Finally, we assume that this log file records a termination symbol whenever a Fix routine terminates. By performing a stack trace, one sees that whenever a Fix routine is called, the clause s that is being repaired by that routine can be deduced from an inspection of the log file H up to that point.

As a consequence, at any intermediate stage in the process of all these fix calls, the original state $A + R$ of the assignment and the random string of bits can be deduced from the current state $A' + R'$ of these objects, plus the history H' up to that point.

Now suppose for contradiction that S is not satisfiable; thus the stack of fix calls can never completely terminate. We trace through this stack for M steps, where M is some large number to be chosen later. After these steps, the random string R has shortened by an amount of Mk; if we set R to initially have length Mk, then the string is now completely empty, $R' = \emptyset$. On the other hand, the history

H' has size at most $O(|S|) + M(k - C + O(1))$, since it takes $|S|$ bits to store the initial clauses in T, $O(|S|) + O(M)$ bits to record all the instances when Step 1 occurs, and every subsequent call to Fix generates a $k - C$-bit number, plus possibly a termination symbol of size $O(1)$. Thus we have a lossless compression algorithm $A + R \mapsto A' + H'$ from $n + Mk$ completely random bits to $n + O(|S|) + M(k - C + O(1))$ bits (recall that A and R were chosen randomly, and independently of each other). But since $n + Mk$ random bits cannot be compressed losslessly into any smaller space, we have the entropy bound

$$n + O(|S|) + M(k - C + O(1)) \geq n + Mk \tag{1}$$

which leads to a contradiction if M is large enough (and if C is larger than an absolute constant). This proves Theorem 1.

> **Remark 1** Observe that the above argument in fact gives an explicit bound on M, and with a small bit of additional effort, it can be converted into a probabilistic algorithm that (with high probability) computes a satisfying assignment for S in time polynomial in $|S|$ and n.

> **Remark 2** One can replace the usage of randomness and Shannon entropy in the above argument with Kolmogorov complexity instead; thus, one sets $A + R$ to be a string of $n + Mk$ bits which cannot be computed by any algorithm of length $n + O(|S| \log |S|) + M(k - C + O(1))$, the existence of which is guaranteed as soon as (1) is violated; the proof now becomes deterministic, except of course for the problem of building the high-complexity string, which by their definition can only be constructed quickly by probabilistic methods. This is the approach taken by Fortnow.

Comments (46)

harrison · 5 August 2009

The link to the Szemeredi regularity lemma is messed up (and I was just about to point out that it's probably the most famous energy-increment argument in additive combinatorics, too!) *[Corrected, thanks – T.]*

Apart from that, wonderful exposition! The insight that the failure of an algorithm on X gives you information about X is, of course, nothing new (it underlies the monotonicity arguments as well as many, many others), but Moser's result is probably the purest use of it I've seen.



Emmanuel Kowalski · 5 August 2009

This is very interesting! I can't help finding some analogy with the problem of constructing large primes, because the application of the Local Lemma seems to have the flavor of a lower-bound sieve argument (getting a probability for the intersection of the complements of events which are close to independent), and we can see the problem of explicit construction of primes as an algorithmic (as opposed to non-constructive) version of sieve...



Anonymous · 5 August 2009

"If something cannot go on forever, it must stop."

I think Markov should be referenced. See "Markov Principle" on wiki.



oguz · 5 August 2009

Dear All,

I am doing my Ph.D. in probability theory and planning to deal with theoretical computer science too. Could you please recommend me some introductory books, survey papers, web blogs in that field?

thanks

nh · 6 August 2009

Very nice exposition, thank you!
I believe that you can slightly improve that compression argument when it comes to determining the value of M. In fact, to store the initial set of unsatisfied clauses, you can just store one bit for each clause, for |S| bits, instead of O(|S| log |S|) bits.
I didn't thoroughly check it, but I believe this gets you to the expected linear running time (as in Moser's result) when you turn the compression argument into a probilistic algorithm.

> **Terence Tao** · 6 August 2009
>
> Hmm, heuristically this seems plausible, but even after storing the initial set T, I think one would need another $O(|S| \log |S|)$ bits to update that set due to the fact that fixing one clause of T may well "accidentally" fix other clauses that were in the queue to be fixed also. It may be though that if one does this carefully enough, one can keep the total entropy cost down to $O(|S|)$, but it would need to be checked.
>
> > **Heinrich** · 8 August 2009
> >
> > The algorithm is "accidentally" fixing many clauses already, namely whenever it reaches Step 1. The $O(1)$ token in the history records this.
> >
> > > **Terence Tao** · 8 August 2009
> > >
> > > Ah yes, you're right, so we can eliminate the log after all. I've amended the text to reflect this.

Anonymous · 6 August 2009

This post is missing an expository tag. *[Well spotted! Corrected, thanks – T.]*

proaonuiq · 6 August 2009

In some cases the hard task is to prove that the monotonicity property, observed experimentally in some cases, holds for all the instances of the problem and it is straightforward to convert this property into a step of the searching algorithm...but the Moser argument seems great.
It is also great how you describe in your posts the hidden mathematical connections: infinite-finite, continuous-discrete, random-structured...

Top Posts « WordPress.com · 6 August 2009

[...] Moser's entropy compression argument There are many situations in combinatorics in which one is running some sort of iteration algorithm to continually &#822 [...] [...]

Foster Boondoggle · 6 August 2009

I'm puzzled by the throwaway comment right at the beginning in parentheses, about not being able to use transfinite induction. Isn't it tautological that an algorithm can't possibly depend on AC? Any method that essentially uses AC cannot possibly have an algorithm (unless by algorithm one includes waving a wand and saying "presto" at a key point).
My layman's understanding of AC is that it plays a role similar to that of an oracle in your previous post – you invoke it to "create" an object (such as a non-principal ultrafilter) that cannot be constructed or exhibited by means involving a finite or countable number of steps – otherwise it would be unnecessary.

> **Qiaochu Yuan** · 6 August 2009
>
> You seem to be using "algorithm" to mean "terminating algorithm." This isn't the case; for example, there are situations in which we can give an algorithm that produces a sequence which

one wants to converge to a point with desirable properties, but it can happen that in order to guarantee convergence we need to index the sequence by an ordinal larger than the natural numbers. Transfinite induction and Zorn's lemma are tools we use to create "extra steps" to guarantee that our algorithm "terminates."



Foster Boondoggle · 6 August 2009

I don't think I mean an algorithm that terminates – e.g., Newton's method for finding $ \sqrt{2}$ is an instance of an algorithm. But isn't it essential to what we mean by $\it algorithm$ that it be amenable to execution? An algorithm that entails AC is, by hypothesis, $ \it not$ executable.



Terence Tao · 7 August 2009

Algorithms which involve AC are, by definition, valid (or "executable", if you will) algorithms in any mathematical universe in which AC holds, and in particular in any model of ZFC. (If one wishes to think algorithmically, one can think of AC as providing a 'choice function' oracle in one's transfinite program. Other axioms of ZFC, such as the axiom of union, axiom of specification, and axiom of infinity, are also needed to set up the architecture needed to "run" a transfinite algorithm, in particular the theory of ordinals, but this is relatively straightforward; see these notes of mine.)

Of course, such an algorithm would not work as stated in the physical universe, but that is not the direct concern of pure mathematics, being instead a topic of computer science (and physics also, to some extent). In particular, transfinite algorithms are not covered by the Church-Turing thesis. (On the other hand, one can convert many (though not all) infinitary algorithms into finitary counterparts which would, in principle, run in the physical universe; see my earlier blog post for further discussion. Furthermore, the independence of AC from the other axioms can often be used to show that a mathematical result obtained using an AC-dependent algorithm, is also necessarily true in the absence of that axiom, even if the algorithm used to prove that result no longer "runs".)

Thus, for instance, the standard greedy algorithm to locate a basis for an arbitrary vector space (which may be infinite dimensional) can be proven to succeed in any ZFC universe, thanks to Zorn's lemma, just as Newton's algorithm to obtain a root of a smooth function from a sufficiently accurate initial guess can be proven to succeed in any model of the reals which obeys the Dedekind completeness axiom (which is part of the standard axiomatisation of the reals, and holds for the standard model of the reals).



Foster Boondoggle · 7 August 2009

Thank you Terry. This has been bothering me ever since I saw a solution (by Stan Wagon) to the infinite version of the prisoners guessing each others' hat colors which involved using a non-principal ultrafilter. (I still think his "solution" was bogus, since the problem statement didn't mention that the prisoners had access to this choice-function oracle.)



andreaberdondini · 31 July 2021

Terence tao I would like to know your answer to this question.

If we roll a ten-sided die a thousand times, what is the average entropy of the generated sequence?

It seems a very easy question, there are 10^1000 possible combinations, each with its own entropy, we just need to take the average.

We get the following value H(X)med=0.9980.

For example, if I toss a coin 3 times I have 2^3=8 possible combinations, each with its own entropy, in this case, H(X)med is:
H(X)med=(H1(X)+H2(X)+H3(X)+H4(X)+H5(X)+H6(X)+H7(X)+H8(X))/8
But, if now we throw the dice once more and calculate the average entropy of 10% of the combinations with less entropy, we get a set equal to the previous 0.1*10^1001 =10^1000.
The entropy value thus obtained is H(X)med2=0.9964.
How can we choose between the two entropy values?
In practice, there are two points of view with which to observe the outcome of the throwing of the dice.
A good way may be to choose the entropy value which multiplied by the length of the sequence gives a smaller value.
0.998*1000=998
0.9964*1001=997.4
Therefore, we should choose the second entropy value.
If we choose the first entropy value, the encoder will take n outputs i.i.d. of the source and maps them on n(H(X)med), in the case of example n=1000.
By choosing the second entropy value, the encoder takes n outputs i.i.d. of the source and maps them to n2(H(X)med2) with n2> n. in the case of example n=1000 and n2=1001.
Starting from an apparently simple question, it is possible to arrive in a not simple situation where we can have different interpretations.
So, I repeat the starting question: If we roll a ten-sided die a thousand times what is the average entropy of the generated sequence?



Heinrich · 8 August 2009

Small typo just below inequality (1), "contradiction of M is large enough" should be ".. if M is ...".
I am slightly confused about the wording surrounding (1). I take it you mean that inequality (1) holds when M is large enough, but this would mean that we can compress completely random bits, a contradiction. I think the naming (1) an "entropy bound" is slightly misleading, or at least has mislead myself, in that (1) does not a priori estimate the entropy of $A + R$ or that of $A' + H'$, but compares the two entropy estimates for large M a posterior.



Terence Tao · 8 August 2009

Oops; I had the inequality (1) the wrong way around. (Both are true, which is where the contradiction comes from, but yes, the attribution was wrong.)
Thanks for the corrections!



Pietro Poggi-Corradini · 8 August 2009

Does the Lovasz local lemma have any connection with the central limit thm?



Mer · 9 August 2009

Hi,
I have a question some how related to this post. Suppose, we have a monotonically decreasing function and we want to find its fixed point, i.e., we have an iterative process with x_{n+1}=f(x_n). Now what is interesting to me is not the fixed point itself but the number of iteration to reach this fixed point given a specific starting point. Is there any way to guess/approximate the number of iterations needed? Even if there is not a general way of approximating the number of iterations for a general monotonic function, a hint to special cases is also appreciated.

Thanks
Mer

 **Anonymous** · 11 August 2009

Is it possible to control the $O(1)$ term more precisely and obtain a better bound on C, say make it arbitrarily small? Is the LLL argument tight for k-sat, i.e. is there an example of an unsatisfiable instance where each clause intersects at most $2^{k – \log_2 e}$ others?

 **Anonymous** · 12 August 2009

Dear Prof. Tao,
in Theorem 1, what do you mean by ” (including s itself)” ?
thanks *[s was supposed to refer to the initial clause; I've adjusted the text a little to reflect this. -T.]*

 **Anonymous** · 12 August 2009

”…it is likely that there will be some non-empty subset T of clauses in S which are not satisfied by S.”
the last S should be A.
thanks *[Corrected, thanks – T.]*

Math World | Moser's entropy compression argument · 14 August 2009
[…] Here is the original post: Moser's entropy compression argument […]

 **piotr** · 7 October 2009

Is the LLL argument tight for k-sat, i.e. is there an example of an unsatisfiable instance where each clause intersects at most $2^{k – \log_2 e}$ others?
Not exactly tight, but quite close. I think this is the best construction of k-sat instances that are unsatisfied and have have variable occurences:
@article{hoory:523,
author = {Shlomo Hoory and Stefan Szeider},
collaboration = {},
title = {A Note on Unsatisfiable k-CNF Formulas with Few Occurrences per Variable},
publisher = {SIAM},
year = {2006},
journal = {SIAM Journal on Discrete Mathematics},
volume = {20},
number = {2},
pages = {523-528},
keywords = {unsatisfiable CNF formulas; NP-completeness; occurrence of variables},
url = {http://link.aip.org/link/?SJD/20/523/1},
doi = {10.1137/S0895480104445745}
}

 **Konstantinos Panagiotou** · 26 January 2010

Thanks for the great post.
Do you think that this approach could be extended to show that any call to FIX terminates after, say, O(log S) recursive calls with high probability? This might not be interesting in the case of the local lemma, but I think it would be great for other applications: think for example of a dynamic data structure in an online setting, where we want to bound the maximum time that a query can take.
I'm not sure if the original proof of Moser gives this result, but Joel Spencer's version of it gives it (if I recall correctly).

 **Shiva** · 26 January 2010

The bitmask representing the initially violate clause is precisely |S| long, so can O(|S|) be simplified to |S| in the last paragraph?

 **AJ** · 18 July 2010

Great exposition!!!

Seminare SS 2011 « ..... DOT's blog ..... · 16 February 2011

[...] Lovász Local Lemma: Hier und hier [...]

Tentative Plans and Belated Updates II | Combinatorics and more · 14 June 2011

[...] These are also good times for other areas of combinatorics. I described some startling developments (e.g., here and here and here) and there is more. There were a few posts (here and here) on the Cup Set Problem. Recently David Bateman and Nets Katz improved, after many years, the Roth-Meshulam bound. See these two posts on Gowers's blog (I,II). Very general theorems discovered independently by David Conlon and Tim Gowers and by Matheas Schacht show that many theorems (such as Ramsey's theorem or Turan's theorem) continue to hold for substructures of sparse random sets. Louis Esperet, Frantisek Kardos, Andrew King, Daniel Kral, and Serguei Norine proved the Lovasz-Palmer conjecture. They showed that every cubic bridgeless graph G has at least perfect matchings. The concept of flag algebras, discovered by Rasborov, is an extremely useful tool for extremal set theory. It has led to solutions of several problems and seems to bring us close to a solution of Turan's Conjecture (which we discussed here and here.) For example, it led to the solution by Hamed Hatami, Jan Hladký, Daniel Král, Serguei Norine, and Alexander Razborov of the question on the maximum number of pentagons in triangle-free graphs. Hamed Hatami found a structure theorem for Boolean functions with coarse thresholds w.r.t. small probabilities. This extends and sharpens results by Ehud Friedgut and Jean Bourgain. I finally caught up (thanks to Reshef Meir) with Moser-Tardos result giving a new algorithmic proof for Lovasz local lemma. Amazing! You can read about it here. [...]

How to measure a vector's quality of having less negative entries. | Help me on math , homework, math activities · 3 February 2012

[...] iterative algorithm which continually improves some vector $mathbf{v}$. Borrowing expressions from Terry Tao's blog, each loop of my algorithm replaces $mathbf{v}$ with some better version $mathbf{v}'$ of [...]

How to measure a vector's quality of having less negative entries. | Q&A System · 12 February 2012

[...] iterative algorithm which continually improves some vector $mathbf{v}$. Borrowing expressions from Terry Tao's blog, every loop of my algorithm replaces $mathbf{v}$ with some an exceptional offer better edition [...]

 **ThankYou** · 26 April 2012

Hi,
could you please check your code for the "entropy bound" again? I cannot see any formula between "...we have the entropy bound" and "which leads to a contradiction...".
Thank You!
[Fixed, thanks – T.]

 **Surya** · 19 October 2012

Dear Prof. Tao
Is there a way to see why the random improvements are producing an assignment even though the Lovasz Local Lemma seems to be capable of producing occurrence of extremely rare events.

Randomised and local choices generally tend to work only when the event of interest occurs with significant probability.
Thank you.



Terence Tao · 19 October 2012

I would classify Moser's algorithm as a semi-random method rather than a random method; the iterative modifications made are not intended to disperse the solution uniformly across the configuration space, but are deliberately engineered to move the proposed solution closer to a true solution by reducing the number of visible "local errors". The randomness is mostly present to avoid getting stuck in some sort of local minimum (cf. annealing methods in variational problems), and of course is also crucial in the theoretical guarantee that a solution must eventually be detected with positive probability.



tap1cse · 11 November 2013

Can you please clarify how that constant term becomes 3?
Thanks in advance.

Quantum Lovasz Local Lemma | Pieces · 22 December 2014
[...] we sketch the core idea of the termination proof for Algorithm 1, and refer interested readers to Terence Tao's blog post, which covers more [...]

Quantum Lovasz Local Lemma | Pieces · 10 June 2015
[...] we sketch the core idea of the termination proof for Algorithm 1, and refer interested readers to Terence Tao's blog post, which covers more detail. Notice that at each call to FIX, we have as input $latex {(R, A, [...]



The logarithmically averaged Chowla and Elliott conjectures for two-point correlations; the Erdos discrepancy problem | What's new · 18 September 2015
[...] reminiscent of the "entropy compression argument" of Moser and Tardos, discussed in this previous post). This argument, which is a simple consequence of the Shannon entropy inequalities, can be viewed [...]



D.S. · 26 July 2018

There is one additional somewhat critical detail missing in this proof. The encoder and decoder for the lossless compression has the formula (that is, the set S of clauses) "built into" it (for example, it needs to consult this to encode/decode some clause as the t-th neighbor of some other clause during a recursive call to Fix). This is not a problem, however, and can be handled in one of two ways: pay for it in the LHS of the key inequality (1) by throwing in an additional |S| k log n term, which will weaken the bound on M by a tiny bit; or (2) use a stronger version of the entropy compression bound asserting that even if the encoder/decoder has a fixed string of this length hardwired into it, n + Mk bits can't be compressed to n + O(|S|) + M(k − C + O(1)) bits on average. (In the Fortnow version, this is captured elegantly if opaquely by writing "Fix a Kolmogorov random string x of length n+sk (random relative to φ, k, s, r, m and n) " [I'm referring to the "relative to φ" part here].



Terence Tao · 7 December 2019

Yes, but no matter how many bits of deterministic information the encoder and decoder have access to, it is still not possible to losslessly encode n random bits into m bits for any fixed $m < n$, simply because the number 2^n of possible values of the input exceeds the number 2^m of the number of possible outputs, and no decoder can overcome the collision that is then forced by the pigeonhole principle regardless of how much deterministic information (independent of the input) that the decoder has access to.

 **The sunflower lemma via Shannon entropy | What's new** · 20 July 2020

[…] The argument bears some resemblance to the "entropy compression method" discussed in this previous blog post; there may be a way to more explicitly express the argument below in terms of that method. (There […]

 **Anonymous** · 7 September 2020

Are there instances where these kind of monotonicity arguments are used to prove lower bounds?

 **Typo?** · 12 April 2021

The end of the paragraph starting with "Each time Step 2 of the Fix algorithm…" should be changed to "…bits of A' on the support of s."
[Corrected, thanks – T.]

 **Entropy Estimation via Two Chains: Streamlining the Proof of the Sunflower Lemma – Theory Dish** · 19 May 2021

[…] We need the following useful lemmas about Shannon entropy. We omit their proofs here as they can be found in Tao's blog post, where many basic properties of Shannon entropy are also discussed. (We also highly recommend Tao's other blog posts about Shannon entropy and the entropy compression argument.) […]